

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	A.Sai Rohit	Reg. No.:	192472144
Section / Batch:	CSE-AI	Date:	

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

I A.Sai Rohit (192472144) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A.Sai Rohit

SECTION A – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

18. Semantic Search System: Create a small document collection and develop semantic search using document embeddings, query embeddings, and retrieval of the most relevant documents.

1. AIM

To develop a semantic search system using Python and the Google Gemini API that converts documents and user queries into embeddings, calculates semantic similarity, and retrieves the most relevant documents.

2. OBJECTIVE

The objectives of this experiment are:

- To create a small collection of documents.
- To generate document embeddings using the Gemini API.
- To generate an embedding for the user's search query.
- To calculate cosine similarity between the query and documents.
- To rank documents based on their similarity scores.
- To retrieve and display the most relevant documents.
- To handle invalid input and API errors.

3. REQUIREMENTS

Software Requirements

- Python 3.x
- Google Gemini API
- NumPy
- Google GenAI Python SDK

Installation

```
pip install google-genai numpy
```

4. API USED

Google Gemini API

Embedding model:

gemini-embedding-001

The Gemini API is used to convert both the documents and the user query into numerical vector representations called embeddings.

5. THEORY

Semantic search retrieves documents based on their meaning rather than only matching exact keywords.

In this system, every document is converted into an embedding vector using the Gemini API. When the user enters a query, the query is also converted into an embedding vector.

The similarity between the query vector and each document vector is calculated using cosine similarity.

The cosine similarity formula is:

$$\text{CosineSimilarity}(A, B) = \frac{A \cdot B}{||A|| ||B||}$$

A higher similarity score indicates that the document is more semantically related to the query.

The documents are then sorted in descending order of their similarity scores, and the top three relevant documents are displayed.

6. ALGORITHM

1. Start the program.
2. Read the Gemini API key from the environment variable.
3. Create a small document collection.
4. Send each document to the Gemini embedding API.
5. Store the generated document embeddings.
6. Accept a search query from the user.
7. Validate the user input.
8. Generate an embedding for the query using the Gemini API.
9. Calculate cosine similarity between the query embedding and every document embedding.
10. Sort the documents according to similarity score.
11. Retrieve the top three relevant documents.
12. Display the document title, similarity score, and content.
13. Handle API errors and invalid input.
14. Repeat the search until the user enters exit.
15. Stop the program.

7. SOURCE CODE

```
import os
import numpy as np
from google import genai

API_KEY = os.getenv("AQ.Ab8RN6K-y28m6kGGawKMi38EVy6Z1Zps0U8oXJ32vLKfvRyHjg")

if not API_KEY:
    print("Error: GEMINI_API_KEY environment variable is not set.")
    exit()

client = genai.Client(api_key=API_KEY)
```

```
documents = [
    {
        "title": "Machine Learning",
        "content": "Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed."
    },
    {
        "title": "Deep Learning",
        "content": "Deep learning uses neural networks with multiple layers to learn complex patterns from large amounts of data."
    },
    {
        "title": "Natural Language Processing",
        "content": "Natural Language Processing allows computers to understand, process and generate human language."
    },
    {
        "title": "Computer Vision",
        "content": "Computer vision enables computers to understand and analyze images and videos using artificial intelligence."
    },
    {
        "title": "Database Management",
        "content": "Database management systems are used to store, organize, retrieve and manage structured data efficiently."
    }
]
```

```
def get_embedding(text):
    try:
        response = client.models.embed_content(
            model="gemini-embedding-001",
            contents=text
        )
        return np.array(response.embeddings[0].values, dtype=np.float32)
    except Exception as e:
        print("API Error:", e)
        return None
```

```
def cosine_similarity(vector1, vector2):
    denominator = np.linalg.norm(vector1) * np.linalg.norm(vector2)

    if denominator == 0:
        return 0.0

    return float(np.dot(vector1, vector2) / denominator)
```

```
def create_document_embeddings():
    print("\nCreating document embeddings...")
```

```

embedded_documents = []

for document in documents:
    embedding = get_embedding(document["content"])

    if embedding is None:
        print("Failed to create embedding for:", document["title"])
        continue

    embedded_documents.append({
        "title": document["title"],
        "content": document["content"],
        "embedding": embedding
    })

    print("Embedded:", document["title"])

return embedded_documents

def semantic_search(query, embedded_documents, top_k=3):
    query_embedding = get_embedding(query)

    if query_embedding is None:
        return []

    results = []

    for document in embedded_documents:
        similarity = cosine_similarity(
            query_embedding,
            document["embedding"]
        )

        results.append({
            "title": document["title"],
            "content": document["content"],
            "similarity": similarity
        })

    results.sort(
        key=lambda x: x["similarity"],
        reverse=True
    )

    return results[:top_k]

def main():
    print("=" * 60)
    print("SEMANTIC SEARCH SYSTEM")

```

```

print("=" * 60)

embedded_documents = create_document_embeddings()

if not embedded_documents:
    print("No documents were successfully embedded.")
    return

while True:
    query = input(
        "\nEnter your search query "
        "(or type 'exit' to quit): "
    ).strip()

    if query.lower() == "exit":
        print("Program terminated.")
        break

    if not query:
        print("Invalid input. Please enter a search query.")
        continue

    results = semantic_search(
        query,
        embedded_documents,
        top_k=3
    )

    if not results:
        print("No search results available.")
        continue

    print("\n" + "=" * 60)
    print("SEARCH RESULTS")
    print("=" * 60)

    for i, result in enumerate(results, start=1):
        print(f"\nRank {i}")
        print("Title:", result["title"])
        print("Similarity Score:", round(result["similarity"], 4))
        print("Content:", result["content"])

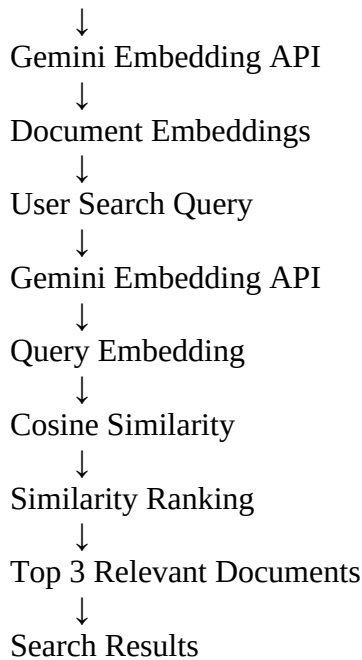
if __name__ == "__main__":
    main()

```

8. WORKING OF THE SYSTEM

The working process is:

Document Collection



9. TEST CASE 1 – NATURAL LANGUAGE PROCESSING

Input

How can computers understand human language?

Output

```
Enter your search query (or type 'exit' to quit): How can computers understand human language?  
...  
=====  
SEARCH RESULTS  
=====
```

Rank 1
Title: Natural Language Processing
Similarity Score: 0.7576
Content: Natural Language Processing allows computers to understand, process and generate human language.

Rank 2
Title: Machine Learning
Similarity Score: 0.6596
Content: Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed.

Rank 3
Title: Computer Vision
Similarity Score: 0.6262
Content: Computer vision enables computers to understand and analyze images and videos using artificial intelligence.

Observation: The Natural Language Processing document is retrieved as the most relevant result because its meaning is closely related to the query.

10. TEST CASE 2 – COMPUTER VISION

Input

How can machines understand pictures?

Output

```
Enter your search query (or type 'exit' to quit): How can machines understand pictures?
...
=====
SEARCH RESULTS
=====

Rank 1
Title: Computer Vision
Similarity Score: 0.7306
Content: Computer vision enables computers to understand and analyze images and videos using artificial intelligence.

Rank 2
Title: Machine Learning
Similarity Score: 0.6631
Content: Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed.

Rank 3
Title: Natural Language Processing
Similarity Score: 0.6402
Content: Natural Language Processing allows computers to understand, process and generate human language.
```

Observation: The Computer Vision document is retrieved as the most relevant document.

11. TEST CASE 3 – MACHINE LEARNING

Input

How do computers learn from data?

Output

```
Enter your search query (or type 'exit' to quit): How do computers learn from data?
=====
SEARCH RESULTS
=====

Rank 1
Title: Machine Learning
Similarity Score: 0.7666
Content: Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed.

Rank 2
Title: Deep Learning
Similarity Score: 0.7069
Content: Deep learning uses neural networks with multiple layers to learn complex patterns from large amounts of data.

Rank 3
Title: Natural Language Processing
Similarity Score: 0.6591
Content: Natural Language Processing allows computers to understand, process and generate human language.
```

Observation: The Machine Learning document receives a high semantic similarity score.

12. TEST CASE 4 – INVALID INPUT

Input

Enter without entering any text

Output

```
Enter your search query (or type 'exit' to quit):
Invalid input. Please enter a search query.
```

Observation: The program validates empty input and prevents invalid searches.

13. TEST CASE 5 – EXIT

Input

Exit

Output

```
Enter your search query (or type 'exit' to quit): exit
Program terminated.
```

14. ERROR HANDLING

The program handles the following cases:

- Missing Gemini API key.
- Gemini API errors.
- Network/API failures.
- Failure to generate document embeddings.
- Failure to generate query embeddings.
- Empty user input.
- No available search results.
- Program termination using exit.

The Gemini API call is protected using exception handling:

```
try:
    response = client.models.embed_content(
        model="gemini-embedding-001",
        contents=text
    )
except Exception as e:
    print("API Error:", e)
```

15. ADVANTAGES

- Searches based on semantic meaning.
- Uses an AI embedding API.
- Provides ranked search results.
- Handles invalid input.
- Handles API errors.
- Easy to extend with additional documents.
- Can be extended to larger document collections and vector databases.

16. APPLICATIONS

Semantic search can be used in:

- Document search systems.

- Question-answering systems.
- Healthcare information retrieval.
- E-learning platforms.
- Customer support systems.
- Enterprise knowledge bases.
- Recommendation systems.
- Retrieval-Augmented Generation (RAG) systems.

17. RESULT

Thus, a **Semantic Search System** was successfully developed using Python and the Google Gemini Embedding API. The system generates embeddings for documents and user queries, calculates cosine similarity, ranks the documents, and retrieves the most relevant documents based on semantic meaning. The system also successfully handles invalid input and API errors.

18. CONCLUSION

The experiment demonstrates how embedding-based semantic search can retrieve relevant information even when the query does not contain the exact keywords present in the documents. The use of Gemini embeddings enables the system to understand the semantic relationship between queries and documents.

23. Compare FAISS and ChromaDB: Develop two versions of a semantic-search system using FAISS and ChromaDB. Compare storage, search, retrieval results, and ease of implementation.

1. AIM

To develop two semantic-search systems using FAISS and ChromaDB, compare their storage, search, retrieval results, and ease of implementation, and identify the suitable use cases for each technology.

2. OBJECTIVES

- To create a small collection of documents.
- To generate document embeddings using the Google Gemini API.
- To store and search embeddings using FAISS.
- To store and search embeddings using ChromaDB.
- To perform semantic search using the same query for both systems.
- To compare search and retrieval results.
- To compare storage mechanisms and ease of implementation.
- To understand the differences between a vector-search library and a vector database.

3. SOFTWARE REQUIREMENTS

- Python 3.x
- Google Gemini API
- NumPy
- FAISS
- ChromaDB
- Google GenAI SDK

4. INSTALLATION

Run the following command in Google Colab:

```
!pip install -q -U google-genai numpy faiss-cpu chromadb
```

5. API AND TECHNOLOGIES USED

Google Gemini API

The Gemini API is used to generate embeddings for documents and user queries.

Embedding model:

gemini-embedding-001
FAISS

FAISS is a library designed for efficient similarity search over dense vectors. It is used in this experiment to store vectors in an index and retrieve the nearest vectors.

ChromaDB

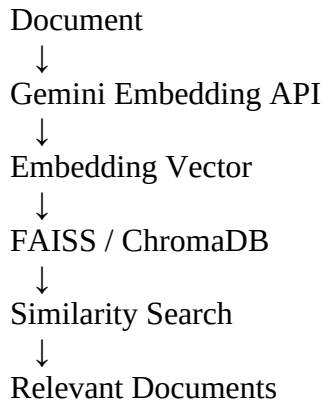
ChromaDB is a vector database that can store documents, embeddings, metadata, and collections and perform similarity-based retrieval.

6. THEORY

Semantic Search

Semantic search retrieves information based on the meaning of a query instead of depending only on exact keyword matching.

The process is:

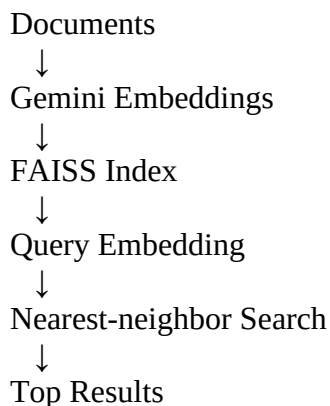


The same embedding model and document collection are used for both systems so that the comparison is fair.

FAISS

FAISS stands for Facebook AI Similarity Search. It provides efficient similarity-search indexes for dense vectors.

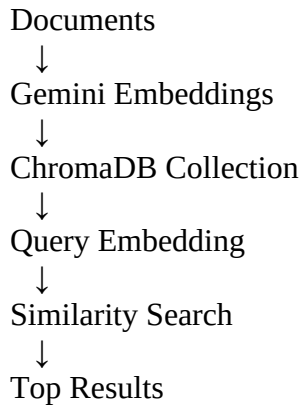
In this experiment:



ChromaDB

ChromaDB is used as a vector database. It can store documents, embeddings, IDs, and metadata in a collection and perform similarity searches.

In this experiment:



7. DOCUMENT COLLECTION

The following documents are used in both systems:

1. Machine Learning
2. Deep Learning
3. Natural Language Processing
4. Computer Vision
5. Database Management

Using the same documents and queries allows a direct comparison.

PART A — FAISS IMPLEMENTATION

8. FAISS SOURCE CODE

```
import os
import numpy as np
import faiss
from google import genai
```

```
API_KEY = os.getenv("AQ.Ab8RN6K-y28m6kGGawKMi38EVy6Z1Zps0U8oXJ32vLKfvRyHjg")
```

```
if not API_KEY:
    print("Error: GEMINI_API_KEY environment variable is not set.")
    exit()
```

```
client = genai.Client(api_key=API_KEY)
```

```
documents = [
    {
        "title": "Machine Learning",
        "content": "Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed."
    },
    {
```

```

        "title": "Deep Learning",
        "content": "Deep learning uses neural networks with multiple layers to learn complex patterns from
large amounts of data."
    },
    {
        "title": "Natural Language Processing",
        "content": "Natural Language Processing allows computers to understand, process and generate
human language."
    },
    {
        "title": "Computer Vision",
        "content": "Computer vision enables computers to understand and analyze images and videos using
artificial intelligence."
    },
    {
        "title": "Database Management",
        "content": "Database management systems are used to store, organize, retrieve and manage structured
data efficiently."
    }
]

```

```

def get_embedding(text):
    try:
        response = client.models.embed_content(
            model="gemini-embedding-001",
            contents=text
        )
        return np.array(response.embeddings[0].values, dtype=np.float32)
    except Exception as e:
        print("API Error:", e)
        return None

```

```

def create_faiss_index():
    vectors = []
    valid_documents = []

    for document in documents:
        embedding = get_embedding(document["content"])

        if embedding is not None:
            vectors.append(embedding)
            valid_documents.append(document)

    if not vectors:
        return None, []

    vectors = np.array(vectors, dtype=np.float32)

    faiss.normalize_L2(vectors)

```

```

dimension = vectors.shape[1]

index = faiss.IndexFlatIP(dimension)
index.add(vectors)

return index, valid_documents

def search_faiss(index, valid_documents, query, top_k=3):
    query_embedding = get_embedding(query)

    if query_embedding is None:
        return

    query_embedding = query_embedding.reshape(1, -1)
    faiss.normalize_L2(query_embedding)

    scores, indices = index.search(query_embedding, top_k)

    print("\nFAISS SEARCH RESULTS")
    print("=" * 60)

    for rank, (score, index_number) in enumerate(
        zip(scores[0], indices[0]), start=1
    ):
        if index_number == -1:
            continue

        print("\nRank:", rank)
        print("Title:", valid_documents[index_number]["title"])
        print("Similarity Score:", round(float(score), 4))
        print("Content:", valid_documents[index_number]["content"])

index, valid_documents = create_faiss_index()

if index is not None:
    query = input("Enter your search query: ").strip()

    if query:
        search_faiss(index, valid_documents, query)
    else:
        print("Invalid input. Please enter a query.")
else:
    print("Unable to create FAISS index.")

```

9. WORKING OF FAISS CODE

1. The document collection is created.
2. Gemini generates an embedding for every document.
3. The embeddings are converted into NumPy arrays.
4. The vectors are normalized.

5. A FAISS IndexFlatIP index is created.
6. The document vectors are added to the index.
7. The user enters a search query.
8. Gemini generates an embedding for the query.
9. FAISS searches for the nearest vectors.
10. The top three documents are returned with similarity scores.

PART B — CHROMADB IMPLEMENTATION

10. CHROMADB SOURCE CODE

```
import os
import chromadb
from google import genai

API_KEY = os.getenv("AQ.Ab8RN6K-y28m6kGGawKMi38EVy6Z1Zps0U8oXJ32vLKfvRyHjg")

if not API_KEY:
    print("Error: GEMINI_API_KEY environment variable is not set.")
    exit()

client = genai.Client(api_key=API_KEY)

documents = [
    {
        "title": "Machine Learning",
        "content": "Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed."
    },
    {
        "title": "Deep Learning",
        "content": "Deep learning uses neural networks with multiple layers to learn complex patterns from large amounts of data."
    },
    {
        "title": "Natural Language Processing",
        "content": "Natural Language Processing allows computers to understand, process and generate human language."
    },
    {
        "title": "Computer Vision",
        "content": "Computer vision enables computers to understand and analyze images and videos using artificial intelligence."
    },
    {
        "title": "Database Management",
        "content": "Database management systems are used to store, organize, retrieve and manage structured data efficiently."
    }
]
```

```

def get_embedding(text):
    try:
        response = client.models.embed_content(
            model="gemini-embedding-001",
            contents=text
        )
        return response.embeddings[0].values
    except Exception as e:
        print("API Error:", e)
        return None

client_db = chromadb.PersistentClient(path="./chroma_db")

collection = client_db.get_or_create_collection(
    name="semantic_search"
)

ids = []
texts = []
embeddings = []
metadatas = []

for i, document in enumerate(documents):
    embedding = get_embedding(document["content"])

    if embedding is not None:
        ids.append(str(i))
        texts.append(document["content"])
        embeddings.append(embedding)
        metadatas.append({"title": document["title"]})

if embeddings:
    collection.upsert(
        ids=ids,
        documents=texts,
        embeddings=embeddings,
        metadatas=metadatas
    )

query = input("Enter your search query: ").strip()

if not query:
    print("Invalid input. Please enter a query.")
else:
    query_embedding = get_embedding(query)

    if query_embedding is not None:
        results = collection.query(
            query_embeddings=[query_embedding],

```

```

        n_results=3
    )

    print("\nCHROMADB SEARCH RESULTS")
    print("=" * 60)

    for i in range(len(results["documents"][0])):
        print("\nRank:", i + 1)
        print(
            "Title:",
            results["metadatas"][0][i]["title"]
        )
        print(
            "Distance:",
            round(results["distances"][0][i], 4)
        )
        print(
            "Content:",
            results["documents"][0][i]
        )
    else:
        print("No documents were embedded successfully.")

```

11. WORKING OF CHROMADB CODE

1. The document collection is created.
2. Gemini generates embeddings for the documents.
3. A persistent ChromaDB client is created.
4. A ChromaDB collection is created.
5. Documents, embeddings, IDs, and metadata are stored.
6. The user enters a search query.
7. Gemini generates the query embedding.
8. ChromaDB performs similarity search.
9. The top three documents are retrieved.
10. The titles, distances, and document contents are displayed.

12. TEST CASE 1

Input

How can computers understand human language?

FAISS Output

Enter your search query: How can computers understand human language?

FAISS SEARCH RESULTS

=====

Rank: 1

Title: Natural Language Processing

Similarity Score: 0.7576

Content: Natural Language Processing allows computers to understand, process and generate human language.

Rank: 2

Title: Machine Learning

Similarity Score: 0.6596

Content: Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed.

Rank: 3

Title: Computer Vision

Similarity Score: 0.6262

Content: Computer vision enables computers to understand and analyze images and videos using artificial intelligence.

ChromaDB Output

... Enter your search query: How can computers understand human language?

CHROMADB SEARCH RESULTS

=====

Rank: 1

Title: Natural Language Processing

Distance: 0.4849

Content: Natural Language Processing allows computers to understand, process and generate human language.

Rank: 2

Title: Machine Learning

Distance: 0.6808

Content: Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed.

Rank: 3

Title: Computer Vision

Distance: 0.7477

Content: Computer vision enables computers to understand and analyze images and videos using artificial intelligence.

Observation

Both FAISS and ChromaDB should retrieve semantically relevant documents. The exact numerical values may differ because the systems report similarity/distance according to their respective search mechanisms.

13. TEST CASE 2

Input

How can machines understand pictures?

FAISS Output

Enter your search query: How can machines understand pictures?

FAISS SEARCH RESULTS

=====

Rank: 1

Title: Computer Vision

Similarity Score: 0.7306

Content: Computer vision enables computers to understand and analyze images and videos using artificial intelligence.

Rank: 2

Title: Machine Learning

Similarity Score: 0.6631

Content: Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed.

Rank: 3

Title: Natural Language Processing

Similarity Score: 0.6402

Content: Natural Language Processing allows computers to understand, process and generate human language.

ChromaDB Output

Enter your search query: How can machines understand pictures?

CHROMADB SEARCH RESULTS

=====

Rank: 1

Title: Computer Vision

Distance: 0.5388

Content: Computer vision enables computers to understand and analyze images and videos using artificial intelligence.

Rank: 2

Title: Machine Learning

Distance: 0.6739

Content: Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed.

Rank: 3

Title: Natural Language Processing

Distance: 0.7197

Content: Natural Language Processing allows computers to understand, process and generate human language.

Observation

Both systems successfully identify documents related to image understanding based on semantic similarity rather than exact keyword matching.

14. TEST CASE 3

Input

How do computers learn from data?

FAISS Output

```
... Enter your search query: How do computers learn from data?

FAISS SEARCH RESULTS
=====

Rank: 1
Title: Machine Learning
Similarity Score: 0.7666
Content: Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed.

Rank: 2
Title: Deep Learning
Similarity Score: 0.7069
Content: Deep learning uses neural networks with multiple layers to learn complex patterns from large amounts of data.

Rank: 3
Title: Natural Language Processing
Similarity Score: 0.6591
Content: Natural Language Processing allows computers to understand, process and generate human language.
```

ChromaDB Output

```
Enter your search query: How do computers learn from data?

CHROMADB SEARCH RESULTS
=====

Rank: 1
Title: Machine Learning
Distance: 0.4668
Content: Machine learning enables computers to learn patterns from data and make predictions without being explicitly programmed.

Rank: 2
Title: Deep Learning
Distance: 0.5862
Content: Deep learning uses neural networks with multiple layers to learn complex patterns from large amounts of data.

Rank: 3
Title: Natural Language Processing
Distance: 0.6818
Content: Natural Language Processing allows computers to understand, process and generate human language.
```

15. TEST CASE 4 — INVALID INPUT

Input

Enter without typing anything

Output

FAISS Output

```
Enter your search query:
Invalid input. Please enter a query.
```

ChromaDB Output

```
Enter your search query:  
Invalid input. Please enter a query.
```

Observation

The program validates user input and prevents an empty search request.

16. STORAGE COMPARISON

FAISS

FAISS primarily provides an in-memory vector index in this implementation. The vectors are added directly to the FAISS index.

It is lightweight and efficient when the application itself manages the surrounding data and persistence.

ChromaDB

ChromaDB provides database-style storage. In this implementation, a persistent client is used:

```
client_db = chromadb.PersistentClient(path="./chroma_db")
```

This allows the collection to be stored locally rather than keeping only an in-memory search index.

17. SEARCH COMPARISON

FAISS

FAISS performs highly efficient nearest-neighbor vector search through specialized indexes.

In this experiment:

```
index = faiss.IndexFlatIP(dimension)
```

is used for inner-product similarity search after vector normalization.

ChromaDB

ChromaDB provides a higher-level interface for vector search. The application can query a collection directly:

```
collection.query(  
    query_embeddings=[query_embedding],  
    n_results=3  
)
```

This reduces the amount of search-index management required in application code.

18. RETRIEVAL RESULTS COMPARISON

Both systems use the same:

- Documents
- Gemini embedding model
- Query
- Number of retrieved results

Therefore, both systems should retrieve semantically similar documents.

The ranking may be similar, while the reported numerical values differ because FAISS reports similarity scores in the selected index configuration, whereas ChromaDB returns distances.

19. EASE OF IMPLEMENTATION

FAISS

Advantages:

- Simple and lightweight.
- Very fast vector search.
- Gives direct control over the search index.
- Suitable for applications where the developer manages storage and metadata.

Disadvantages:

- More application-level code is required for document and metadata management.
- Persistence needs to be handled separately.
- Developers need to understand FAISS indexes and vector dimensions.

ChromaDB

Advantages:

- Higher-level vector database interface.
- Stores documents, embeddings, IDs, and metadata together.
- Supports persistent collections.
- Simple query interface.
- Convenient for RAG and document-retrieval applications.

Disadvantages:

- More components than a basic FAISS index.
- Slightly more overhead than a minimal in-memory FAISS implementation.
- Database management features may be unnecessary for very small applications.

20. COMPARISON TABLE

Feature	FAISS	ChromaDB
Type	Vector similarity-search library	Vector database
Main Purpose	Fast vector search	Vector storage and retrieval
Storage	Primarily index-based	Database/collection-based
Persistence	Needs explicit handling	Supported through persistent client
Metadata Management	Usually handled separately	Built into collection workflow
Search	Very fast and flexible	Simple high-level vector search
Ease of Basic Implementation	Moderate	Easy
Control Over Indexes	High	Higher-level abstraction

21. ADVANTAGES OF FAISS

- High-performance similarity search.
- Efficient for large vector collections.
- Provides multiple indexing strategies.
- Lightweight.
- Gives developers detailed control over vector search.

22. ADVANTAGES OF CHROMADB

- Easy-to-use vector database interface.
- Stores embeddings and documents together.
- Supports metadata.
- Supports persistent storage.
- Useful for RAG and knowledge-retrieval applications.

23. APPLICATIONS

FAISS Applications

- Image similarity search.
- Recommendation systems.
- Large-scale vector search.
- Duplicate detection.
- Semantic retrieval.

ChromaDB Applications

- RAG systems.
- AI chatbots.

- Document search.
- Knowledge bases.
- Question-answering systems.
- LLM applications.

24. FINAL COMPARISON

FAISS is more suitable when high-performance vector similarity search and direct control over indexes are the main requirements.

ChromaDB is more suitable when an application requires convenient storage of documents, embeddings, metadata, and persistent collections along with vector retrieval.

Therefore:

FAISS → High-performance vector search and control

ChromaDB → Easier vector database and application integration

25. RESULT

Thus, two semantic-search systems were successfully developed using FAISS and ChromaDB. Both systems used Gemini-generated embeddings to perform semantic retrieval. The systems were compared based on storage, search, retrieval results, and ease of implementation.

26. CONCLUSION

FAISS and ChromaDB both provide effective solutions for semantic search, but they serve different purposes. FAISS provides a lightweight and highly controllable similarity-search solution, while ChromaDB provides a higher-level vector database approach with document, metadata, and persistence support. The appropriate technology depends on the requirements of the application.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
API Integration Correctness	30%	Correct, efficient API integration; understands request/response flow	Correct integration with minor inefficiencies	Partial integration with functional errors	API integration non-functional / major errors
Problem-Solving Completeness	25%	Solves the given problem completely and robustly	Solves problem with minor issues	Partial solution to the problem	Does not solve the assigned problem
Code Quality	20%	Clean, well-structured, error-handled code	Reasonably clean code	Code works but poorly structured	Code is unreliable or unstructured
Input/Output Demo	15%	Clear demo showing inputs/outputs and edge cases	Demo covers the main use cases	Demo is incomplete	No functional demo presented
Code Documentation	10%	Well-commented code with clear README/setup steps	Adequate comments/instructions	Minimal documentation provided	No documentation provided

Marks Evaluation:

Question No.	API Integration Correctness(30%)	Problem-Solving Completeness (25%)	Code Quality(20%)	Input/Output Demo(15%)	Code Documentation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						